

Hadoop: Data Flow, I/O, Data Integrity, Compression, Serialization

Your Name

September 18, 2025

Agenda

- 1 Introduction to Hadoop
- 2 Hadoop Data Flow
- 3 HDFS and Block Storage
- 4 MapReduce Process
- 5 Hadoop I/O Overview
- 6 Data Integrity Techniques
- 7 Compression in Hadoop
- 8 Serialization Mechanisms

What is Hadoop?

- Open-source framework for distributed storage and processing
- Developed by the Apache Software Foundation
- Handles big data across clusters of computers
- Components include HDFS, YARN, MapReduce, and others
- Designed to be scalable and fault-tolerant
- Written in Java but supports many programming languages
- Works well with structured and unstructured data
- Enables batch processing for large-scale datasets

Hadoop Ecosystem

- HDFS: Hadoop Distributed File System
- YARN: Yet Another Resource Negotiator
- MapReduce: Data processing engine
- Hive: SQL-like interface for Hadoop
- Pig: High-level scripting for data flows
- HBase: NoSQL database for Hadoop
- Sqoop: Imports data from RDBMS
- Flume: Collects and loads log data

Hadoop Data Flow

- Describes movement of data in Hadoop jobs
- Input data is loaded from HDFS
- Split into logical InputSplits
- Map tasks process InputSplits into key-value pairs
- Shuffle sort phase groups keys from mappers
- Reduce tasks aggregate values by key
- Final results are written to HDFS
- Involves disk I/O, network I/O, and memory buffers

HDFS Data Flow

- Files are split into blocks (default 128MB)
- Each block is replicated across multiple DataNodes
- Namenode stores metadata for blocks and files
- Datanodes store actual block data
- Client requests go through Namenode
- Data is written in parallel to DataNodes
- Checksum is computed for each block
- Replication ensures fault tolerance

MapReduce Data Flow

- JobTracker splits the job into map tasks
- InputFormat defines data splits and parsing
- Mapper processes input into intermediate key-value pairs
- Combiner (optional) aggregates mapper output locally
- Partitioners distribute output by key to reducers
- Shuffle stage transfers data to reducers
- Reducers process and write final output
- OutputFormat writes results to HDFS

Hadoop I/O

- Handles reading/writing data efficiently
- Uses InputFormat and OutputFormat interfaces
- Supports compression for input/output
- Data is serialized using Writable or other formats
- Supports custom data types
- Buffered reads and writes to reduce I/O cost
- Compatible with multiple storage formats
- Key for optimizing MapReduce performance

InputFormat in Hadoop

- Splits input data into InputSplits
- Provides RecordReader to parse splits
- Common formats include:
 - TextInputFormat: Reads lines of text
 - KeyValueTextInputFormat: Parses key-value from lines
 - SequenceFileInputFormat: For binary key-value files
 - DBInputFormat: Reads from relational databases
- Supports custom InputFormat classes

OutputFormat in Hadoop

- Writes output from reducer to storage
- Defines how key-value pairs are stored
- Common formats:
 - `TextOutputFormat`: Writes plain text
 - `SequenceFileOutputFormat`: Binary format
 - `MultipleOutputs`: Writes to multiple files
 - `LazyOutputFormat`: Avoids creating empty files
- Custom `OutputFormats` allow for flexibility

Writable and Combiner

- Writable is Hadoop's serialization format
- Efficient for Java-based data types
- Examples: IntWritable, Text, LongWritable
- Must implement readFields and write methods
- Combiner reduces intermediate data volume
- Acts like a mini-reducer
- Improves performance by reducing network I/O
- Not guaranteed to be called; must be idempotent

Data Integrity in Hadoop

- Ensures data correctness and reliability
- Prevents corruption during read/write
- HDFS generates checksums for each block
- Validates checksum during read
- Automatic re-replication on failure
- Client retries failed operations
- Detects silent corruptions
- Essential for big data reliability

Data Integrity Techniques

- Checksum stored per block (512 bytes by default)
- Verified at read time by the client
- Namenode monitors DataNode health
- Block reports sent from DataNodes
- Corrupted blocks are flagged
- Missing blocks trigger re-replication
- HDFS balances replicas across nodes
- Heartbeat checks ensure cluster health

Compression in Hadoop

- Compression reduces storage and speeds up data transfer.
- Reduces disk storage space
- Lowers I/O and network transfer time
- Compression codecs define algorithms
- Transparent to the user in most cases
- Hadoop supports pluggable codecs
- Improves job execution time for large data

Types of Compression

- Input Compression: Compressed files as input
- Map Output Compression: Compresses intermediate data
- Output Compression: Final reducer output compressed
- Block Compression: Compresses blocks of records
- Record Compression: Compresses individual records
- Codec configuration via mapred-site.xml
- Supported by Input/OutputFormat classes
- Saves bandwidth in distributed processing

Compression Codecs in Hadoop

- Gzip: High compression, slower speed
- Bzip2: Better compression, very slow
- Snappy: Very fast, low compression
- LZO: Fast compression/decompression
- DefaultCodec: Generic wrapper
- Custom codecs can be implemented
- Codec class used in job configuration
- Tradeoff: speed vs compression ratio

Serialization in Hadoop

- Converts objects into byte stream
- Essential for data exchange in distributed systems
- Writable is Hadoop's native serialization format
- Serialization affects network and disk I/O
- Good serialization improves job speed
- Must be efficient and compact
- Supports complex data types
- Enables data persistence and transmission

Writable Interface

- Base interface for serialization in Hadoop
- Must implement `Writable` interface
- Implements `readFields()` and `write()`
- Lightweight and fast
- Used in key-value pair classes
- Includes types like `Text`, `IntWritable`, `FloatWritable`
- Allows custom data structures
- Critical for Hadoop I/O performance

Other Serialization Formats

- Avro: Supports schemas and versioning
- Thrift: Used for RPC communication
- Protocol Buffers: Developed by Google
- More portable than Writable
- Language-independent
- Supports nested and complex types
- Schema evolution for future-proofing
- Common in enterprise-grade Hadoop systems

Summary

- Hadoop processes large-scale data efficiently
- Data flows through HDFS and MapReduce pipeline
- I/O managed using InputFormat, OutputFormat, Writable
- Data integrity ensured with checksums and replication
- Compression improves performance and storage
- Writable enables fast native serialization
- Avro, Thrift, and Protobufs add flexibility
- Understanding internals helps optimize jobs